Documentación Proyecto 2

- Sebastián León - 202510399
- Juan David Saldaña - 202511865
- Sebastián Martínez - 202510157

1. Análisis y revisión de modelo:

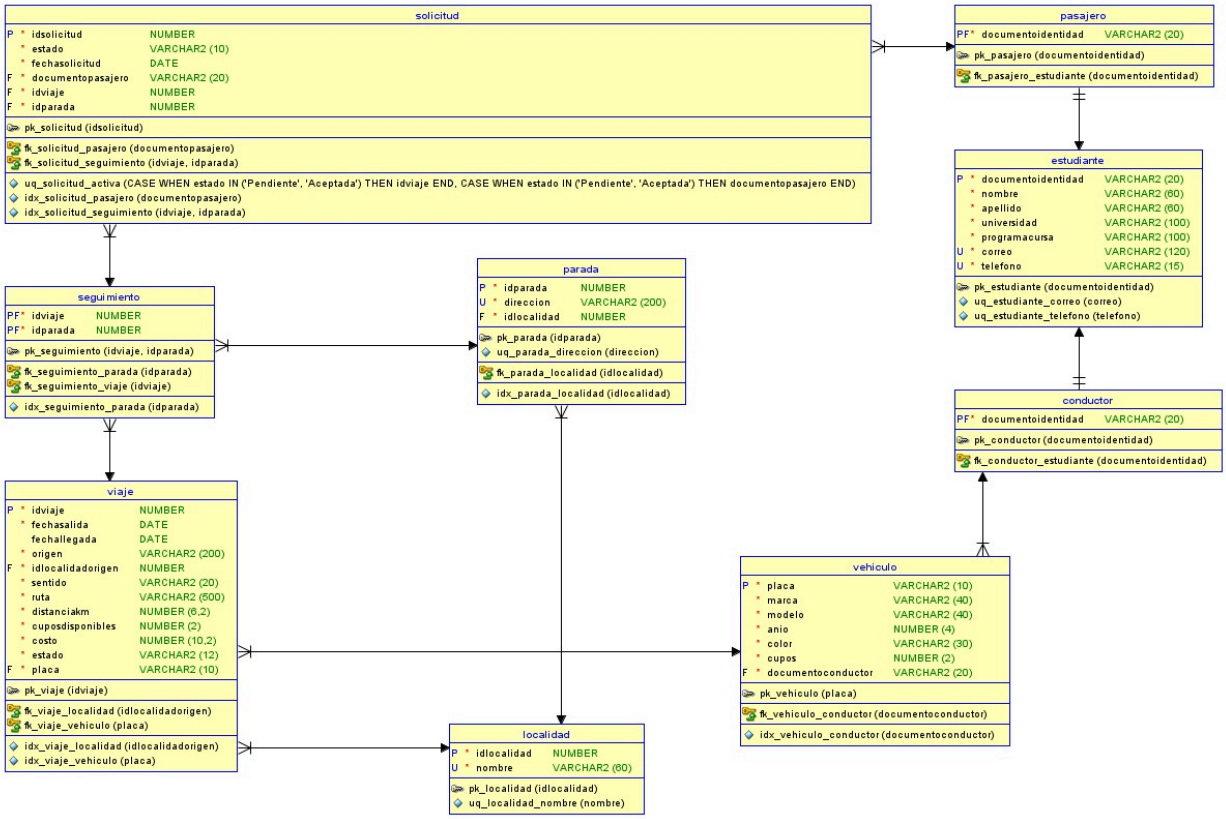
El sistema está diseñado para administrar de forma automatizada la oferta y demanda de viajes compartidos en el ámbito universitario. Su arquitectura se fundamenta en un modelo relacional robusto donde la entidad central es estudiante, la cual se subdivide mediante herencia en conductor y pasajero. De este modo, un estudiante se registra opcionalmente como conductor o pasajero. Cada conductor posee uno o más vehículos $(1: N)$, los cuales a su vez realizan múltiples viajes $(1: N)$. A nivel geográfico y de infraestructura, una localidad ubica varias paradas y actúa como el origen de diversos trayectos $(1: N)$. Para estructurar la ruta con precisión, la entidad asociativa seguimiento vincula los viajes con sus respectivas paradas $(1: N)$. Finalmente, los pasajeros realizan solicitudes de cupo $(1: N)$, especificando el punto exacto de recogida mediante la relación entre la solicitud y el seguimiento $(1: N)$.

Para garantizar la integridad operativa y la consistencia de los datos, el sistema implementa un conjunto estricto de triggers en la base de datos. En cuanto a la gestión de viajes, se valida que todo registro nuevo nazca obligatoriamente en estado "Programado" con una fecha de salida estrictamente futura, y que sus cupos disponibles nunca superen la capacidad máxima configurada en el vehículo asignado. Asimismo, se establecen restricciones de inmutabilidad para viajes que ya se encuentren completados o cancelados, el registro automático de la marca de tiempo de llegada al finalizar el recorrido, un margen mínimo de dos horas de anticipación para cancelaciones, y una actualización en cascada que cancela automáticamente todas las solicitudes pendientes o aceptadas asociadas a un viaje que ha sido cancelado.

Por otro lado, la gestión de solicitudes exige que toda nueva petición inicie en estado "Pendiente" exclusivamente sobre viajes programados que cuenten con cupos disponibles, prohibiendo de forma taxativa que un conductor solicite un cupo dentro de su propio viaje. Las solicitudes aceptadas únicamente pueden ser modificadas para pasar a un estado de cancelación con al menos dos horas de anticipación a la salida del trayecto. Su ciclo de vida actualiza de manera dinámica los cupos del viaje: disminuyendo la disponibilidad al ser aceptadas y restituyendo el cupo de forma automática en caso de cancelación o cambio de estado.

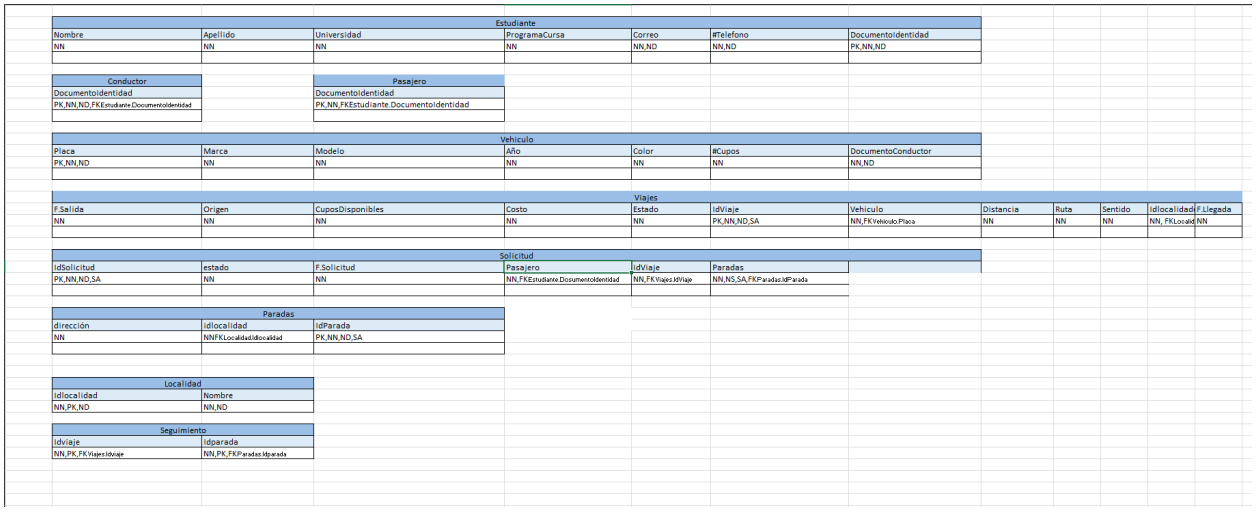
Entre el modelo relacional inicial y el esquema final mostrado, se realizaron cambios importantes que quitaron restricciones innecesarias entre pasajeros y viajes, eliminaron relaciones repetidas,

corrigieron errores en las restricciones de unicidad de las llaves foráneas y definieron claves primarias compuestas en tablas como Seguimiento.Este esquema está en Cuarta Forma Normal (4FN) porque cumple con la Forma Normal de Boyce-Codd (BCNF) y eliminó todas las dependencias multivaluadas no triviales al separar atributos independientes en tablas distintas, como vehículos, conductores, solicitudes, paradas y seguimientos, evitando así redundancias por duplicación de listas de valores.



Entidad (lado uno)	Entidad (lado muchos)	Cardinalidad	Nombre (de uno a muchos)	Nombre (de vuelta)
estudiante	conductor	1 : 0..1	se registra como	es un
estudiante	pasajero	1 : 0..1	se registra como	es un
conductor	vehiculo	1 : N	posee	pertenece a
vehiculo	viaje	1 : N	realiza	usa
localidad	parada	1 : N	ubica	esta en
localidad	viaje	1 : N	es origen de	sale de
viaje	seguimient	1 : N	tiene	pertenece a
parada	seguimient	1 : N	se incluye en	usa
pasajero	solicitud	1 : N	realiza	es hecha por
seguimient	solicitud	1 : N	recibe	pide recoger en

Indicaciones del modelo entidad relación



Estudiante						
Nombre	Apellido	Universidad	ProgramaCursa	Correo	#Telefono	DocumentoIdentidad
NN	NN	NN	NN	NN,ND	NN,ND	PK,NN,ND

Conductor
DocumentoIdentidad
PK,NN,ND,FK.Estudiante.DocumentoIdentidad

Pasajero
DocumentoIdentidad
PK,NN,FKEstudiante.DocumentoIdentidad

Vehiculo						
Placa	Marca	Modelo	Año	Color	#Cupos	DocumentoConductor
PK,NN,ND	NN	NN	NN	NN	NN	NN,ND

Viajes											
F.Salida	Origen	CuposDisponibles	Costo	Estado	IdViaje	Vehiculo	Distancia	Ruta	Sentido	Idlocalidad	F.Llegada
NN	NN	NN	NN	NN	PK,NN,ND,SA	NN,FK Vehiculo.Placa	NN	NN	NN	NN, FK.Localid	NN

Solicitud					
IdSolicitud	estado	F.Solicitud	Pasajero	IdViaje	Paradas
PK,NN,ND,SA	NN	NN	NN,FK.estudiante.DocumentoIdentidad	NN,FK.Viajes.IdViaje	NN,NS,SA,FKParadas.IdParada

Paradas		
dirección	Idlocalidad	IdParada
NN	NNFK.Localidad.Idlocalidad	PK,NN,ND,SA

Localidad	
Idlocalidad	Nombre
NN,PK,ND	NN,ND

Seguimiento	
Idviaje	Idparada
NN,PK,FK.Viajes.Idviaje	NN,PK,FKParadas.Idparada

Normalización actualizada

2. Implementación esquema - Población tablas:

El proyecto se despliega y verifica mediante una serie de scripts secuenciales organizados en un orden estricto de dependencias. El proceso inicia con 00_eliminar_tablas.sql, el cual limpia de forma destructiva el esquema existente utilizando la opción de purga para evitar registros en la papelera de reciclaje. Posteriormente, 01_crear_tablas.sql construye el esquema relacional declarando claves primarias, foráneas, restricciones de unicidad, validaciones de verificación (CHECK), valores por defecto e índices condicionales avanzados para las solicitudes que se encuentran activas.

Esto en tres scripts independientes que deben ejecutarse respetando la jerarquía de las entidades. El script 02_poblar_base.sql inserta el volumen inicial correspondiente a localidades, paradas, estudiantes, conductores, pasajeros y vehículos. A continuación, 03_poblar_viajes.sql registra los trayectos y su respectiva asociación con las paradas a través de la tabla de seguimiento. Finalmente, 04_poblar_solicitudes.sql incorpora las peticiones de los pasajeros con sus diferentes estados y fechas para habilitar las pruebas del sistema.

Para asegurar el cumplimiento de las reglas de negocio, el sistema implementa un conjunto de triggers que regulan de forma automatizada tanto los viajes como las solicitudes. Los disparadores de viajes controlan que toda nueva inserción nazca en estado "Programado" con fechas futuras, validan que los cupos disponibles no superen la capacidad del vehículo, gestionan las transiciones de estado de forma inmutable, registran la hora de llegada y aplican una cancelación en cascada sobre las solicitudes asociadas cuando un viaje es cancelado. Por su parte, los disparadores de solicitudes exigen que las nuevas peticiones inicien en estado "Pendiente" sobre viajes programados con disponibilidad y evitan que un conductor solicite cupo en su propio trayecto, controlando además los cambios de estado permitidos y actualizando dinámicamente los cupos del viaje de manera centralizada.

El script 06_triggers.sql implementa un conjunto de validaciones y automatizaciones críticas a nivel de base de datos para las entidades de viaje y solicitudes. Por un lado, se asegura que un viaje o solicitud cumpla con los estados iniciales correctos (como 'Programado' o 'Pendiente'), valide restricciones de fechas futuras, verifique la existencia y capacidad del vehículo asociado, impida que un conductor solicite un cupo en su propio viaje y controle las estrictas reglas de transición de estados y plazos de cancelación con al menos dos horas de anticipación. Por otro lado, hace uso de un trigger compuesto (compound trigger) para actualizar automáticamente los cupos disponibles de los viajes y cancelar de manera masiva las solicitudes pendientes o aceptadas asociadas a un viaje cuando este es cancelado.

3. Diseño sentencias sql:

Ahora se va a presentar la documentación de los requerimientos funcionales de consulta, explicando de qué va cada una, con las tablas y joins utilizados.

RFC1 – Consultar demanda por localidad para un período de tiempo específico:

Para este requerimiento, la consulta se encarga de calcular el volumen total de solicitudes de cupo organizadas por el nombre descriptivo de cada localidad dentro de un rango de fechas específico. Las tablas involucradas en esta operación son Solicitud (que documenta las peticiones y sus respectivas fechas), Parada (que vincula cada solicitud con una ubicación concreta) y Localidad (que guarda el nombre oficial de la zona geográfica). Se utiliza un INNER JOIN, ya que es necesario garantizar que solo se procesen y contabilicen aquellas solicitudes que tengan una correspondencia estricta y válida con una parada registrada y, por lo tanto, con una localidad existente en el sistema, excluyendo cualquier registro huérfano.

RFC2 – Consultar ingresos estimados por localidad y período:

Esta consulta funciona calculando los ingresos totales estimados generados, la cantidad de viajes completados y las solicitudes aceptadas por cada localidad en un rango de fechas específico, permitiendo visualizar reportes completos incluso en periodos sin actividad. Las tablas involucradas son Localidad (como entidad base para la agrupación), Viaje (para filtrar los trayectos completados en las fechas indicadas y sumar sus costos) y Solicitud (para contabilizar las peticiones con estado aceptado). El tipo de join utilizado es LEFT JOIN, el cual se implementa de manera estratégica para garantizar que todas las localidades del sistema aparezcan en el reporte final (mostrando ceros en los ingresos y conteos cuando corresponda), cumpliendo adecuadamente con los escenarios de prueba donde no existen resultados en el rango de fechas consultado.

RFC3 – Consultar viajes por conductor a través del tiempo:

El propósito de esta consulta es mostrar la evolución histórica y detallada de los viajes realizados por los conductores, permitiendo filtrar opcionalmente por un conductor específico, una localidad y un rango de fechas. Las tablas utilizadas son Viaje (que contiene los datos temporales, estado y costos del trayecto), Vehiculo (para vincular el viaje con el automóvil correspondiente), Conductor y Estudiante (para extraer la identificación y el nombre completo del conductor), y Localidad (para obtener el nombre legible de la zona de origen). El tipo de join utilizado es INNER JOIN, porque se necesita relacionar de forma rigurosa y directa cada viaje

con su vehículo, el conductor propietario, sus datos personales y la localidad de origen para desplegar la información completa y consistente de los trayectos válidos.

RFA1 – Tablero de control para la información de los conductores:

Para este requerimiento, la consulta funciona extrayendo el nombre completo y el documento de identidad de los usuarios registrados como conductores para alimentar los filtros o la vista general del tablero de control que consolida métricas clave como el total de pasajeros transportados, tiempos, ingresos y distancias. Las tablas utilizadas son conductor (para identificar el rol específico de conducción) y estudiante (que almacena los datos personales de los usuarios). El tipo de join empleado es un JOIN (o INNER JOIN), vinculado a través de la columna documentoidentidad, el cual es necesario para garantizar que solo se listen y evalúen aquellos conductores que poseen un registro de estudiante válido y existente dentro del sistema.

RFA2 – Tablero de control para la información de pasajeros:

Esta consulta opera recuperando la identificación y el nombre completo de los pasajeros para configurar las opciones del tablero analítico encargado de mostrar los viajes tomados, las zonas frecuentes, los gastos totales y los promedios de tiempo y distancia. Las tablas involucradas son pasajero (que delimita el rol de los usuarios que solicitan servicios de transporte) y estudiante (para rescatar los atributos de nombre y apellido). El tipo de join utilizado es un JOIN (o INNER JOIN), conectado mediante el campo común documentoidentidad, lo que permite cruzar de forma estricta los datos de la persona con su rol institucional de pasajero para estructurar correctamente la información del tablero.

4. Aplicación:

Primero que todo, para el ingreso a la aplicación es necesario saber que el usuario y la contraseña a escribir para poder ingresar al usuario, es el correo y el documento de identidad respectivamente. Esto gracias a una función que fue creada con ese propósito.

Además, para la realización de la aplicación se sabe que están los usuarios de conductor y pasajero. Cada uno de estos tiene requerimientos distintos, por ende se debe separar lo que puede hacer cada tipo de usuario. Para esto se realizó la creación de una autorización mediante authorization esquema, en el cual se dividió los requerimientos necesarios.

Cabe recalcar que algunos de los requerimientos no funcionaron, como es el caso del RFC2, el cual presentó muchos problemas debido a que no se proceso bien los filtros de fecha y los elementos de la página, generando que cuando Oracle intenta transformar de condiciones nulas,

generando un error. Así mismo, hubo ciertos requerimientos los cuales la página funcionaba perfecto, sin embargo, al momento de ejecutarla mostraba resultados nulos o vacíos, tales como el RF3 y el RFA2, los cuales quedaron vacíos, sin embargo si sintaxis era correcta, y su interfaz también fue correcta

De manera curiosa, al ejecutar las funciones fuera de la aplicación están funcionando correctamente, sin embargo, al ejecutar estas mismas en el oracle apex, estas no funcionan, por lo que dedujimos que el problema radicó en la manera de plasmarlo en la aplicación.

Para crear la aplicación correspondiente se tuvo en cuenta los requerimientos funcionales ya mencionados, se intentó hacer una página por requerimiento, por lo que se puede decir que uno de los mayores problemas a los que nos enfrentamos fue al uso de la interfaz de Oracle Apex, porque como se mencionó anteriormente, muchos de los códigos aprobados con antelación en la sección de comandos dieron los resultados esperados, pero a la hora de retratarlos en la aplicación estos fallaban, si desean probarlos, la mayoría de los códigos de consulta se encuentran dentro de los archivos sql.

5. Pruebas:

Prueba RF3: Se prueba creación de viaje, por parte del conductor. Se llenan los espacios correspondientes y se espera que aparezcan dentro de la base de datos en la tabla viajes.



VIAJE

Columns | Data | Indexes | Constraints | Grants | Statistics | Triggers | Dependencies | DDL | Sample Queries

+ Insert Row | Columns... | Filter... | Count Rows | Load Data | Download | Refresh

IDVIAJE	FECHASALIDA	FECHALLEGADA	ORIGEN	IDLOCALIDADORIG	SENTIDO	RUTA	DISTANCIAKM	CUPOSDISPONIBLE	COSTO	ESTADO	PLACA
80	9/30/2026		Calle 63 con Ca...	3	Hacia universidad	Calle 63 con Ca...	11.2	4	6500	Cancelado	LPJ493
122	10/16/2026	10/16/2026	Universidad de ...	8	Desde universi...	Universidad de ...	12	0	6500	Completado	LPJ493
123	10/19/2026	10/19/2026	Parque de la 93	3	Hacia universidad	Parque de la 93...	9.7	0	6000	Completado	LPJ493
178	10/28/2026	10/30/2026	Universidad de ...	3	Desde universi...	Caracas con 80	5	4	5000	Programado	LPJ493
197	10/27/2026		Universidad de ...	3	Desde universi...	Caracas - Auto...	7	2	3500	Programado	LPJ493
38	9/10/2026	9/10/2026	Calle 63 con Ca...	3	Hacia universidad	Calle 63 con Ca...	12.4	1	7000	Completado	LPJ493

Comprobante RF-3

Prueba RF-6: Se prueba la búsqueda de los viajes en la base de datos con sus correspondientes pasajeros, siendo filtrados como lo deseé el conductor. Para esto se modifican los filtros dispuestos.

RF6

Search: Idviaje | Reset

Idviaje contains 170

Idviaje	Fechasalida	Fechallegada	Origen	Sentido	Estado Viaje	Placa	Cuposdisponibles	Costo	Documento Pasajero	Nombre Pasajero	Apellido Pasajero	Correo Pasajero
170	11/6/2026		Portal Américas	Hacia universidad	Programado	GLJ195	2	6000	1004170694	Simón	Rojas Álvarez	simon.rojas@uniandes.edu.co
170	11/6/2026		Portal Américas	Hacia universidad	Programado	GLJ195	2	6000	1094267632	Juan	Suárez Bernal	juan.suarez@uniandes.edu.co

1 - 2

VIAJE

Columns | Data | Indexes | Constraints | Grants | Statistics | Triggers | Dependencies | DDL | Sample Queries

+ Insert Row | Columns... | Filter... | Count Rows | Load Data | Download | Refresh

IDVIAJE	FECHASALIDA	FECHALLEGADA	ORIGEN	IDLOCALIDADORIG	SENTIDO	RUTA	DISTANCIAKM	CUPOSDISPONIBLE	COSTO	ESTADO	PLACA
170	11/6/2026		Portal Américas	13	Hacia universidad	Portal América...	9.8	2	6000	Programado	GLJ195

SOLICITUD

Columns | Data | Indexes | Constraints | Grants | Statistics | Triggers | Dependencies | DDL | Sample Queries

+ Insert Row | Columns... | Filter... | Count Rows | Load Data | Download | Refresh

IDSOLICITUD	ESTADO	FECHASOLICITUD	DOCUMENTOPASAJERO	IDVIAJE	IDPARADA
564	Pendiente	10/17/2026	1072870673	170	26
585	Pendiente	10/18/2026	1020654844	170	26
595	Aceptada	10/18/2026	1004170694	170	26
523	Aceptada	10/16/2026	1094267632	170	30
653	Pendiente	10/20/2026	1068761242	170	30

Comprobantes RF-6

Prueba RF-9: Se busca los viajes que aparecen en la base de datos, para esto es posible filtrarlos de distintas maneras como quiera el pasajero.

RF9

Search: Idviaje

Reset

Idviaje contains 178

Idviaje	Fechasalida	Origen	Sentido	Ruta	Distanciakm	Cuposdisponibles	Costo	Estado	Placa	Marca	Modelo	Anio	Color	Documento Conductor	Nombre Conductor	Telefono Conductor
178	10/28/2026	Universidad de los Andes	Desde universidad	Caracas con 80	5	4	5000	Programado	LPJ493	Renault	Sandero	2016	Plata	1010834547	Natalia García Ramos	3047543956

1 - 1

Comprobante RF-9

6. Extra:

Durante la realización del proyecto, hubo varios problemas con respecto al manejo de la aplicación. Por ejemplo, Sebastián León durante la parte del desarrollo de la aplicación en un momento dejó de funcionar la página, dejando un mensaje de error y que no se podría ingresar al trabajo ni modificarlo por un periodo de tiempo prolongado. Adjunto pruebas:

